Date:15/10/2026

CO1-AT3-CODING CHALLENGE

Course Code: DSA0308

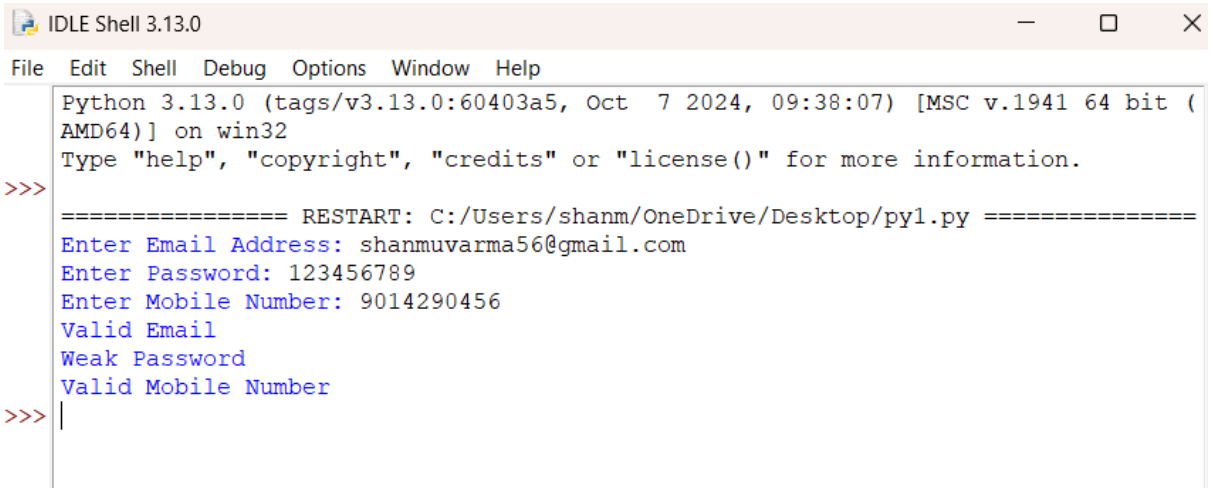
Course name: Natural Language Processing

Reg No: 192324315L **Name**: A. Gayathri Shanmukhi

1. Intelligent Email and Password Validator using Regular Expressions program

```
import re
email = input("Enter Email Address: ")
password = input("Enter Password: ")
mobile = input("Enter Mobile Number: ")
email_pattern = r'^[A-Za-z][A-Za-z0-9._]*@[A-Za-z]+\.(com|org|edu|net|in)$'
password_pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@#$%&!]).{8,}$'
mobile_pattern = r'^[6-9]\d{9}$'
if re.match(email_pattern, email):
    print("Valid Email")
else:
    print("Invalid Email")
if re.match(password_pattern, password):
    print("Strong Password")
else:
    print("Weak Password")
if re.match(mobile_pattern, mobile):
    print("Valid Mobile Number")
else:
    print("Invalid Mobile Number")
```

Output



2. Design a Finite State Automata (FSA) Simulator program

```
states = input("Enter states: ").split()
alphabet = input("Enter input alphabet: ").split()
initial = input("Enter initial state: ")
final = input("Enter final states: ").split()
transition = {}
for state in states:
    for symbol in alphabet:
        next_state = input(f"Transition({state}, {symbol}): ")
        transition[(state, symbol)] = next_state
n = int(input("Enter number of strings: "))
for _ in range(n):
    string = input("Enter input string: ")
    current = initial
    valid = True
    for symbol in string:
        if symbol not in alphabet:
            valid = False
            break
        current = transition[(current, symbol)]
```

```
    if valid and current in final:
        print("Accepted")
    else:
        print("Rejected")
```

Output

```
Python 3.13.0 (tags/v3.13.0:60403a5, Oct  7 2024, 09:38:07) [MSC v.1941 64 bit (
AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
================ RESTART: C:/Users/shanm/OneDrive/Desktop/py1.py ================
Enter states: q0 q1 q2
Enter input alphabet: a b
Enter initial state: q0
Enter final states: q2
Transition(q0, a): q1
Transition(q0, b): q0
Transition(q1, a): q1
Transition(q1, b): q2
Transition(q2, a): q1
Transition(q2, b): q0
Enter number of strings: 3
Enter input string: abaab
Accepted
Enter input string: ab
Accepted
Enter input string: abg
Rejected
```

3. Smart Pattern Matching Engine using Regular Expressions program

```
import re
text = input("Enter text: ")
print("1. Search Word")
print("2. Search Date")
print("3. Search Phone Number")
print("4. Search Hashtag")
print("5. Search Mention")
print("6. Search Prefix")
print("7. Search Suffix")
choice = input("Enter your choice: ")
patterns = {
    "1": r'\b\w+\b',
    "2": r'\b\d{2}/\d{2}/\d{4}\b',
    "3": r'\b[6-9]\d{9}\b',
    "4": r'#\w+',
    "5": r'@\w+',
```

```
    "6": r'\bnatural\w*',
    "7": r'\w+ing\b'
}
if choice in patterns:
    result = re.findall(patterns[choice], text, re.IGNORECASE)
    if result:
        print("Matching Patterns:", result)
    else:
        print("No Matching Patterns Found")
else:
    print("Invalid Choice")
```

output

```
>>>
================ RESTART: C:/Users/shanm/OneDrive/Desktop/py1.py ================
Enter text: Meeting on 12/09/2026 Call 9876543210 #NLP @OpenAI natural language
processing
1. Search Word
2. Search Date
3. Search Phone Number
4. Search Hashtag
5. Search Mention
6. Search Prefix
7. Search Suffix
Enter your choice: 2
Matching Patterns: ['12/09/2026']
>>>
```